The Journal of Open Source Software

pycellga: A Python package for improved cellular genetic algorithms

Sevgi Akten Karakaya [1] **and Mehmet Hakan Satman** [2]

1 Department of Informatics, Istanbul University, Istanbul, Turkey **2** Department of Econometrics, Istanbul University, Istanbul, Turkey

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Josh Borrow

Reviewers:

- @jmejia8
- @jbussemaker

Submitted: 14 August 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

pycellga is a Python package that implements cellular genetic algorithms (CGAs) for optimizing complex problems. CGAs combine the principles of cellular automata and traditional genetic algorithms, utilizing a spatially structured population organized in a grid-like topology. This structure allows each individual to interact only with its neighboring individuals, promoting diversity and maintaining a balance between exploration and exploitation during the optimization process.

The package is designed to be user-friendly, with a straightforward installation process and comprehensive documentation. Researchers and practitioners in fields such as operations research, artificial intelligence, and machine learning can leverage pycellga to tackle complex optimization challenges effectively. The integration of cellular automata with genetic algorithms in pycellga represents a significant advancement in the field of evolutionary computation, offering increased flexibility and adaptability compared to traditional methods. pycellga also includes machine-coded operators with byte implementations, developed by (Satman, 2013). Additionally, it features Alpha-male CGA, Machine-Coded Compact CGA, and Improved CGA with Machine-Coded Operators for real-valued optimization problems (Karakaya & Satman, 2024).

Introduction

Optimization problems are a fundamental aspect of various scientific and engineering fields, involving the search for the best solution among a large set of possible options. Genetic algorithms (GAs) have been widely used to address these problems due to their robustness and adaptability. Inspired by the process of natural selection, GAs operate on a population of potential solutions, applying operators such as selection, crossover, and mutation to evolve the population toward better solutions over successive generations (Goldberg, 1989; Holland, 1975).

Despite their effectiveness, traditional GAs face challenges, particularly in maintaining diversity within the population and avoiding premature convergence to suboptimal solutions (Goldberg & Deb, 1991). To mitigate these issues, researchers have developed cellular genetic algorithms (CGAs), which introduce a spatial structure to the population (Manderick & Spiessens, 1991; Whitley, 1993). In a CGA, individuals are placed on a grid, and interactions are restricted to neighboring individuals. This localized interaction promotes diversity and enables a more thorough exploration of the solution space.

pycellga is a Python package designed to efficiently implement CGAs. By integrating the principles of cellular automata with genetic algorithms, pycellga offers a robust framework for tackling complex optimization problems. The pycellga package is designed to handle a

DRAFT

wide range of optimization problems, including binary, real-valued, and permutation-based challenges, making it a versatile tool for diverse applications in evolutionary computation. The package includes several built-in functions for initialization, selection, crossover, mutation, and evaluation, as well as customization options to cater to different needs. This flexibility allows researchers and practitioners to apply CGAs to a wide range of problems with ease (Karakaya & Satman, 2024).

By providing a comprehensive toolkit for CGAs, `pycellga` aims to advance the field of evolutionary computation and equip researchers with the tools needed to solve increasingly complex optimization problems effectively. The integration of cellular automata with genetic algorithms in `pycellga` represents a significant advancement, offering greater flexibility and adaptability compared to traditional methods (Eiben & Smith, 2015; Karakaya & Satman, 2024; Michalewicz, 1996). `pycellga` has machine coded operators with byte implementations, developed by (Satman, 2013). Additionally, it features Alpha-male CGA, developed based on insights from (Satman & Akadal, 2019), Machine-Coded Compact CGA, developed based on insights from (Satman & Akadal, 2020), and Improved CGA with Machine-Coded Operators (Karakaya & Satman, 2024). The improved cellular genetic algorithm uses machine-coded operators specifically designed for real-valued optimization problems. This method stands out by employing byte-based operators, which are crafted to efficiently process numerical data in terms of memory usage.

State of the field

There are several existing software packages that implement genetic algorithms, such as DEAP and PyGAD. However, most of these packages do not specifically focus on the integration of cellular automata with genetic algorithms, except for JCell, which is a Java implementation of CGAs (Alba & Dorronsoro, 2008). `pycellga` addresses this gap by offering a specialized toolkit for CGAs, leveraging the strengths of both methodologies. `pycellga` includes machine-coded operators with byte-level implementations. Additionally, it features methods such as Alpha-male CGA, Machine-Coded Compact CGA, and Improved CGA with Machine-Coded Operators.

Statement of need

The need for `pycellga` arises from the increasing complexity of optimization problems and the limitations of traditional genetic algorithms in handling these complexities. By incorporating cellular automata, `pycellga` introduces localized interactions and diversity within the population, which can lead to more effective and efficient solutions. This package is particularly useful for researchers and practitioners who wish to explore advanced genetic algorithm techniques. `pycellga` includes machine-coded operators with byte-level implementations developed by (Satman, 2013). Additionally, it features Alpha-male CGA, Machine-Coded Compact CGA, and an Improved CGA with Machine-Coded Operators for real-valued optimization problems (Karakaya & Satman, 2024).

Installation and basic usage

`pycellga` can be downloaded and installed by using the following command:

```
pip install pycellga
```

Usage Examples

In this section, we'll explain cga method in the optimizer and provide an example of how to use it. The package includes various ready-to-use crossover and mutation operators, along

DRAFT

with real-valued, binary, and permutation functions that you can run directly. Examples for other methods are available in the `example` folder, while an example for cga is provided below.

cga (Cellular Genetic Algorithm)

cga is a type of genetic algorithm where the population is structured as a grid (or other topologies), and each individual interacts only with its neighbors. This structure helps maintain diversity in the population and can prevent premature convergence. To specialize the CGA for real-valued optimization problems, ICGA (Improved CGA) with machine-coded representation can be used, applying byte operators. The encoding and decoding of numbers follow the IEEE 754 standard for floating-point arithmetic, yielding better results for continuous functions.

Example Problem

Suppose we have a problem that we want to minimize using a Cellular Genetic Algorithm (CGA). The problem is defined as a simple sum of squares function, where the goal is to find a chromosome (vector) that minimizes the function.

The sum of squares function computes the sum of the squares of each element in the chromosome. This function reaches its global minimum when all elements of the chromosome are equal to 0. The corresponding function value at this point is 0.

ExampleProblem Class

Here's how we can define this problem in Python using the `ExampleProblem` class:

```python
from mpmath import power as pw
import pycellga

class ExampleProblem:

    def __init__(self):
        pass

    def f(self, x):

        return sum(pw(xi, 2) for xi in x)
```

Usage:

```python
result = pycellga.optimizer.cga(
    n_cols = 5,
    n_rows = 5,
    n_gen = 100,
    ch_size = 5,
    gen_type = pycellga.optimizer.GeneType.REAL,
    p_crossover = 0.9,
    p_mutation = 0.2,
    problem = ExampleProblem(),  # Replace with a real problem instance as needed
    selection = pycellga.optimizer.TournamentSelection,
    recombination = pycellga.optimizer.ByteOnePointCrossover,
    mutation = pycellga.optimizer.ByteMutationRandom,
    mins = [-32.768] * 5,  # Minimum values for each gene
    maxs = [32.768] * 5    # Maximum values for each gene
)
print("Best solution:", result[1], "\nBest solution chromosome:", result[0])
```

DRAFT

```
# The result is
# Best solution: 0.0
# Best solution chromosome: [0.0, 0.0, 0.0, 0.0, 0.0]

# Note that the result could be different because the algorithm includes randomness.
```

References

Alba, E., & Dorronsoro, B. (2008). *Cellular genetic algorithms*. Springer. https://doi.org/10.1007/978-0-387-77610-1_1

Eiben, A. E., & Smith, J. E. (2015). *Introduction to evolutionary computing*. Springer. https://doi.org/10.1007/978-3-662-44874-8

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley. https://doi.org/10.5860/choice.27-0936

Goldberg, D. E., & Deb, K. (1991). A comparative analysis of selection schemes used in genetic algorithms. *Foundations of Genetic Algorithms*, *1*, 69–93. https://doi.org/10.1016/b978-0-08-050684-5.50008-2

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. MIT Press. ISBN: 9780262581110

Karakaya, S. A., & Satman, M. H. (2024). An improved cellular genetic algorithm with machine-coded operators for real-valued optimisation problems. *Journal of Engineering Research and Applied Science*, *13*(1), 2500–2514.

Manderick, B., & Spiessens, P. (1991). The genetic algorithm and the structure of the fitness landscape. In R. K. Belew & L. B. Booker (Eds.), *Proceedings of the 4th international conference on genetic algorithms (ICGA)* (pp. 143–150). Morgan Kaufmann.

Michalewicz, Z. (1996). *Genetic algorithms + data structures = evolution programs* (3rd ed.). Springer. https://doi.org/10.1007/978-3-662-03315-9

Satman, M. H. (2013). Machine coded genetic algorithms for real parameter optimization problems. *Gazi University Journal of Science*, *26*(1), 85–95.

Satman, M. H., & Akadal, E. (2019). Performance comparison of the specialized alpha male genetic algorithm with some evolutionary algorithms. *Trakya University Journal of Social Science*, *21*(1), 55–82. https://doi.org/10.26468/trakyasobed.452095

Satman, M. H., & Akadal, E. (2020). Machine coded compact genetic algorithms for real parameter optimization problems. *Alphanumeric Journal*, *8*(1), 43–58. https://doi.org/10.17093/alphanumeric.576919

Whitley, D. (1993). Cellular genetic algorithms. In S. Forrest (Ed.), *Proceedings of the 5th international conference on genetic algorithms (ICGA)* (p. 658). Morgan Kaufmann.

DRAFT